



**Student Name:** Bhavya  
**Branch:** CSE(AI & ML)  
**Semester:**4  
**Subject Name:** Database Management System

**UID:** 24BAI70791  
**Section/Group:** 24AIT\_KRG-1/G2  
**Subject Code:** 24CSH-298

## Experiment – 10

### 1. Aim of the Session

To design a trigger that automatically implements the functionality of a primary key, ensuring unique identification of records without manual intervention.

### 2. Software Requirements

#### Database Management System:

- Oracle Database Express Edition (Oracle XE)
- PostgreSQL Database

#### Database Administration Tool / Client Tool:

- Oracle SQL Developer (for Oracle XE)
- pgAdmin (for PostgreSQL)

### 3. Objectives

To create a database trigger that automatically enforces primary key constraints or generates unique key values, replicating the functionality of a stored procedure.

#### 4. Procedure of the Experiment

1. **Create the table** in the database with required fields and define a primary key column (or leave it for trigger-based assignment).
2. **Create a sequence** (or auto-increment logic) to generate unique values for the primary key.
3. **Design a trigger function** that runs before an INSERT operation.
4. Inside the trigger, **check if the primary key value is NULL** or not provided.
5. If NULL, **assign a unique value using the sequence or counter logic**.
6. **Create and attach the trigger** to the table for INSERT events.
7. **Insert sample records** without specifying the primary key value.
8. **Verify the output** by checking the table to ensure each record has a unique automatically generated primary key.

#### 5. Practical / Experiment Steps

##### **PROGRAM CODE:**

```
CREATE TABLE employee (  
    emp_id INTEGER PRIMARY KEY,  
    name VARCHAR(100),  
    department VARCHAR(50)  
);
```

```
CREATE SEQUENCE emp_seq START 1;
```

```
CREATE OR REPLACE FUNCTION emp_trigger_function()  
RETURNS TRIGGER AS $$  
  
BEGIN  
  
    IF NEW.emp_id IS NULL THEN
```

```
NEW.emp_id := nextval('emp_seq');

END IF;

RETURN NEW;

END;

$$ LANGUAGE plpgsql;

CREATE TRIGGER emp_trigger
BEFORE INSERT ON employee
FOR EACH ROW
EXECUTE FUNCTION emp_trigger_function();

INSERT INTO employee (name, department) VALUES ('John', 'HR');
INSERT INTO employee (name, department) VALUES ('Alice', 'IT');

SELECT * FROM employee;
```

## 6. Input / Output Details and Screenshot

```
2
3 CREATE TABLE employee (
4   emp_id INTEGER PRIMARY KEY,
5   name VARCHAR(100),
6   department VARCHAR(50)
7 );
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 1 secs 49 msec.

8

9 `CREATE SEQUENCE emp_seq START 1;`

Data Output Messages Notifications

CREATE SEQUENCE

Query returned successfully in 246 msec.

10

```
11 CREATE OR REPLACE FUNCTION emp_trigger_function()  
12 RETURNS TRIGGER AS $$  
13 BEGIN  
14 IF NEW.emp_id IS NULL THEN  
15     NEW.emp_id := nextval('emp_seq');  
16 END IF;  
17 RETURN NEW;  
18 END;  
19 $$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 101 msec.

Query Query History

20

```
21 CREATE TRIGGER emp_trigger  
22 BEFORE INSERT ON employee  
23 FOR EACH ROW  
24 EXECUTE FUNCTION emp_trigger_function();
```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 1 secs 19 msec.

```

24 EXECUTE FUNCTION emp_trigger_function();
25
26 INSERT INTO employee (name, department) VALUES ('John', 'HR');
27 INSERT INTO employee (name, department) VALUES ('Alice', 'IT');

```

Data Output Messages Notifications

INSERT 0 1

Query returned successfully in 188 msec.

```

28
29 SELECT * FROM employee;

```

Data Output Messages Notifications

Showing rows: 1 to 2 | Page No: 1 of 1

|   | emp_id<br>[PK] integer | name<br>character varying (100) | department<br>character varying (50) |
|---|------------------------|---------------------------------|--------------------------------------|
| 1 | 1                      | John                            | HR                                   |
| 2 | 2                      | Alice                           | IT                                   |

## 7. Learning Outcome

After completing this experiment, the learner will be able to:

- Understand the purpose and working of database triggers.
- Implement automated primary key functionality using triggers.
- Ensure data integrity without manual key assignment.
- Apply trigger-based automation in real-world enterprise applications such as Amazon, Flipkart, and Oracle.